Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Регламент нагрузочного прогона

Стенд, данные, доказательства и критерии · версия 1.0.0

30 октября 2026

© Alexander Yurasov

Содержание

1 Регламент нагрузочного прогона 3

2 1. Стенд 4

3 2. Данные 5

4 3. Ограничение ресурсов 6

5 4. Запуск API 7

6 5. Сбор доказательств 8

7 6. Критерии 10

8 7. Узкое место 11

9 8. Порядок серии 12

1 Регламент нагрузочного прогона

Порядок, по которому выполняется и оформляется любой нагрузочный прогон версии 1.0.0 (E51-01): сайзинг, пики, длительная работа, отказы и проверка исправлений. Модель нагрузки и профили — `docs/performance/workday-model.md`, диагностика по симптомам — `docs/runbooks/diagnostics.md`. Прогон без доказательств из раздела «Сбор доказательств» не принимается: в отчёт попадает только то, что можно воспроизвести командой.

2 1. Стенд

Перед прогоном записывается конфигурация — она входит в отчёт первой таблицей.

ПАРАМЕТР	КАК УЗНАТЬ
Процессор и память узла	`nproc`, `free -g`, `lscpu | grep 'Model name'`
Ядра API и PostgreSQL	`taskset -pc <pid>` — при ограничении ресурсов (раздел 3)
Процессов API, пул на процесс	`WEB_CONCURRENCY`, `DB_POOL_SIZE` или `connection_limit`
PostgreSQL	`SELECT version()`, `SHOW shared_buffers`, `SHOW max_connections`
Версия и коммит	`git rev-parse --short HEAD`, `version` в `server/package.json`
Данные	Численность, лет истории, размер базы — раздел 2

Генератор работает на том же узле на отдельных ядрах; его загрузка видна в `gen_cores` сборщика ресурсов, и прогон, где генератор занял свои ядра полностью, не принимается — задержки в нём завышены генератором.

3 2. Данные

Отдельная база `vacation_load`; рабочая база и база тестов не используются.

```
cd server
export DATABASE_URL=postgresql://vacation:vacation@localhost:5432/vacation_load
npx prisma migrate deploy
npm run db:seed:load -- --employees 2000 --history 5
psql "$DATABASE_URL" -c 'VACUUM ANALYZE' -c "SELECT
pg_size_pretty(pg_database_size(current_database()))"
```

Наполнение детерминировано и очищает таблицы стенда перед записью: каждый прогон серии начинается с одинаковых данных. Прогоны, которые меняют данные (согласование, открытие кампании), запускаются на свежем наполнении.

4 3. Ограничение ресурсов

Для сайзинга (E51-02) API и PostgreSQL получают заданное число ядер на одном узле — так проверяется конфигурация «всё на одном узле» без виртуальных машин. Дочерние процессы наследуют привязку: процессы кластера API — от главного, новые соединения PostgreSQL — от `postmaster`.

```
# PostgreSQL: главный процесс и уже запущенные дочерние — на ядро 1
PG=$(head -1 /var/lib/postgresql/16/main/postmaster.pid)
for p in $PG $(pgrep -P "$PG"); do taskset -pc 1 "$p"; done

# API: на ядро 0; WEB_CONCURRENCY=auto видит только доступные ядра
taskset -c 0 env WEB_CONCURRENCY=auto node dist/main.js

# Генератор и сборщик ресурсов — на остальные ядра
taskset -c 2,3 node ../tools/load/run.mjs ...
```

Конфигурация «2 vCPU» — API и PostgreSQL на общих ядрах `0,1`; «4 vCPU» — без ограничения. Объём памяти ограничивается параметрами, а не привязкой: `shared_buffers`, `work_mem` и число процессов API.

5 4. Запуск API

```
cd server && npm run build
DATABASE_URL=postgresql://vacation:vacation@localhost:5432/vacation_load \
JWT_SECRET=<32+ символа> PORT=8097 WEB_CONCURRENCY=auto \
LOG_FORMAT=json REQUEST_LOG=slow SLOW_REQUEST_MS=500 SLOW_QUERY_MS=0 \
node dist/main.js > api.log 2>&1
```

ПЕРЕМЕННАЯ	ПРОГОН ИЗМЕРЕНИЯ	ПРОГОН ДИАГНОСТИКИ
`REQUEST_LOG`	`slow`	`slow` или `all` — `all` пишет каждый запрос и сам нагружает диск
`SLOW_REQUEST_MS`	500	300 — порог требования к чтению
`SLOW_QUERY_MS`	0 — выключен: замер каждого запроса к БД стоит процессора	100 — запросы к БД с идентификатором запроса API
`NODE_OPTIONS`	—	`--cpu-prof --cpu-prof-dir=prof` — профиль процессора каждого процесса при остановке

Почта на стенде не настроена: письма остаются в очереди `email_outbox`, а скорость разбора очереди измеряется отдельно (отчёт пиков).

6 5. Сбор доказательств

Перед прогоном — обнулить статистику БД и запустить сборщик ресурсов:

```
psql "$DATABASE_URL" -c 'CREATE EXTENSION IF NOT EXISTS pg_stat_statements' \
  -c 'SELECT pg_stat_statements_reset()' -c 'SELECT pg_stat_reset()'
node tools/load/sample.mjs --db "$DATABASE_URL" --interval 5 --out resources.csv &
```

`pg_stat_statements` требует `shared_preload_libraries=pg_stat_statements` — в `docker-compose.yml` версии 1.0.0 он включён; на стенде без контейнеров — строкой в `postgresql.conf` и перезапуском PostgreSQL.

Сборщик пишет каждые 5 секунд: загрузку узла, занятые ядра отдельно у API, PostgreSQL и генератора, память процессов API (сумму и наибольший), соединения с БД и активные из них. Прогон:

```
node tools/load/run.mjs --url http://127.0.0.1:8097 --profile average --population 2000 --
out result.json | tee result.md
```

После прогона:

```
kill %1                                                    # сборщик ресурсов
psql "$DATABASE_URL" -f deploy/postgres/top-queries.sql > top-queries.txt
curl -s http://127.0.0.1:8097/metrics > metrics.txt        # счётчики одного
процесса кластера
jq -r 'select(.msg=="slow request") | .route' api.log | sort | uniq -c | sort -rn | head
# медленные маршруты
jq -r 'select(.msg=="slow query") | "\(.model).\(.operation)\(.query // "")"' api.log | sort
| uniq -c | sort -rn | head
```

ДОКАЗАТЕЛЬСТВО	ФАЙЛ	ЧТО ПОКАЗЫВАЕТ
Итог генератора	`result.md`, `result.json`	Задержки по операциям, ступени, лента по минутам
Ресурсы	`resources.csv`	Что упирается первым: ядра API, ядра PostgreSQL, соединения, память
Запросы к БД	`top-queries.txt`	Самые дорогие по суммарному и среднему времени, таблицы с полным просмотром
Журнал API	`api.log`	Медленные запросы API и БД с идентификатором запроса; ошибки 5xx со стеком

ДОКАЗАТЕЛЬСТВО	ФАЙЛ	ЧТО ПОКАЗЫВАЕТ
Профиль процессора	`prof/*.cpuprofile`	Где тратит время процесс API — открывается в Chrome DevTools, вкладка Performance

Итог генератора (`result.json`) и выдержки из остального — в каталоге отчёта `docs/performance/results/` ; журналы и профили целиком в репозиторий не кладутся.

7 6. Критерии

Прогон пройден, если выполнены все требования модели (`workday-model.md`, раздел «Требования к задержкам и ошибкам»): p95 чтения ≤ 300 мс, записи ≤ 500 мс, вход ≤ 1000 мс, 0 ошибок, 0 сбоев потока событий, генератор не упёрся в свои ядра. Для `stress` критерий другой — ступень, на которой требования нарушены впервые; запас — отношение предыдущей ступени к ×1. Для `soak` дополнительно: p95 последнего часа не больше p95 первого часа на 20 % и больше, наибольший RSS процесса API не растёт после первого часа.

8 7. Узкое место

Узкое место признаётся только при трёх доказательствах сразу (E51-06):

1. **Симптом** — рост задержки операции в итоге генератора или в ленте по минутам;
2. **Причина** — совпадающая по времени метрика: занятые ядра в `resources.csv`, запрос в `top-queries.txt` или медленный запрос в журнале с тем же маршрутом, функция в профиле процессора;
3. **Воспроизведение** — команда, которая показывает проблему отдельно от прогона: запрос с `EXPLAIN (ANALYZE, BUFFERS)`, вызов API через `curl` с замером, короткий прогон с `--stages`.

Исправление проверяется повтором того же прогона на тех же данных; в отчёт попадают оба результата — до и после.

9 8. Порядок серии

1. `smoke` — стенд и контракт: 0 ошибок, все операции отвечают.
2. `average` и `spike` для 1000 и 2000 сотрудников на ограниченных ресурсах — сайзинг (E51-02).
3. `stress` на выбранной конфигурации — запас.
4. Пики сервера на фоне `average` — отчёт пиков (E51-03).
5. `soak` 8 часов — в фоне, пока готовятся отчёты (E51-04).
6. Отказы на фоне `average`: перезапуск процесса API, PostgreSQL, переключение на реплику (E51-07).
7. Повтор шагов 2–3 после исправлений узких мест (E51-06).